

MATH5320 Homework Week 9

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

V0 = 15_000_000
sigma = 0.30
r = 0.05
T = 5.0
target_survival = 0.96368

# Merton functions

def d2(B):
    B = np.asarray(B, dtype=float)
    return (np.log(V0 / B) + (r - 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))

def d1(B):
    return d2(B) + sigma * np.sqrt(T)

def survival(B):
    return norm.cdf(d2(B)) # Q(V_T >= B)

def equity(B):
    return V0 * norm.cdf(d1(B)) - B * np.exp(-r * T) * norm.cdf(d2(B))

def debt(B):
    return V0 - equity(B)

# Solve for B* such that survival(B*) = target_survival
z = norm.ppf(target_survival)
B_star = V0 * np.exp(-z * sigma * np.sqrt(T) + (r - 0.5 * sigma**2) * T)

# Values at B*
d2_star = d2(B_star)
d1_star = d1(B_star)
s_star = survival(B_star)
E_star = equity(B_star)
D_star = debt(B_star)

print("=== Matched face value ===")
print(f"B* = {B_star:,.2f}")
print(f"d2(B*) = {d2_star:.6f}")
print(f"d1(B*) = {d1_star:.6f}")
print(f"Survival(B*) = {s_star:.6f}")
print(f"Equity(B*) = {E_star:,.2f}")
print(f"Debt(B*) = {D_star:,.2f}")

# Grid for plots
B_grid = np.linspace(0.5e6, 30e6, 1200)

s_grid = survival(B_grid)
E_grid = equity(B_grid)
```

```

D_grid = debt(B_grid)

plt.figure(figsize=(8, 5))
plt.plot(B_grid / 1e6, s_grid, label='Survival probability $s(B)$')
plt.axvline(B_star / 1e6, linestyle='--', label=f'$B^* \approx \{B\_star/1e6\}$')
plt.axhline(target_survival, linestyle=':', label=f'Target survival = {target_survival}')
plt.xlabel('Face value B (millions)')
plt.ylabel('Survival probability')
plt.title('Merton model: survival probability vs face value')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(B_grid / 1e6, E_grid / 1e6, label='Equity value $E_0(B)$')
plt.axvline(B_star / 1e6, linestyle='--', label=f'$B^* \approx \{B\_star/1e6\}$')
plt.scatter([B_star / 1e6], [E_star / 1e6], zorder=3, label=f'$E_0(B^*)$')
plt.xlabel('Face value B (millions)')
plt.ylabel('Equity value (millions)')
plt.title('Merton model: equity value vs face value')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

=
plt.figure(figsize=(8, 5))
plt.plot(B_grid / 1e6, D_grid / 1e6, label='Debt value $D_0(B)$')
plt.axvline(B_star / 1e6, linestyle='--', label=f'$B^* \approx \{B\_star/1e6\}$')
plt.scatter([B_star / 1e6], [D_star / 1e6], zorder=3, label=f'$D_0(B^*)$')
plt.xlabel('Face value B (millions)')
plt.ylabel('Debt value (millions)')
plt.title('Merton model: debt value vs face value')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

=== Matched face value ===

$B^* = 4,612,960.81$

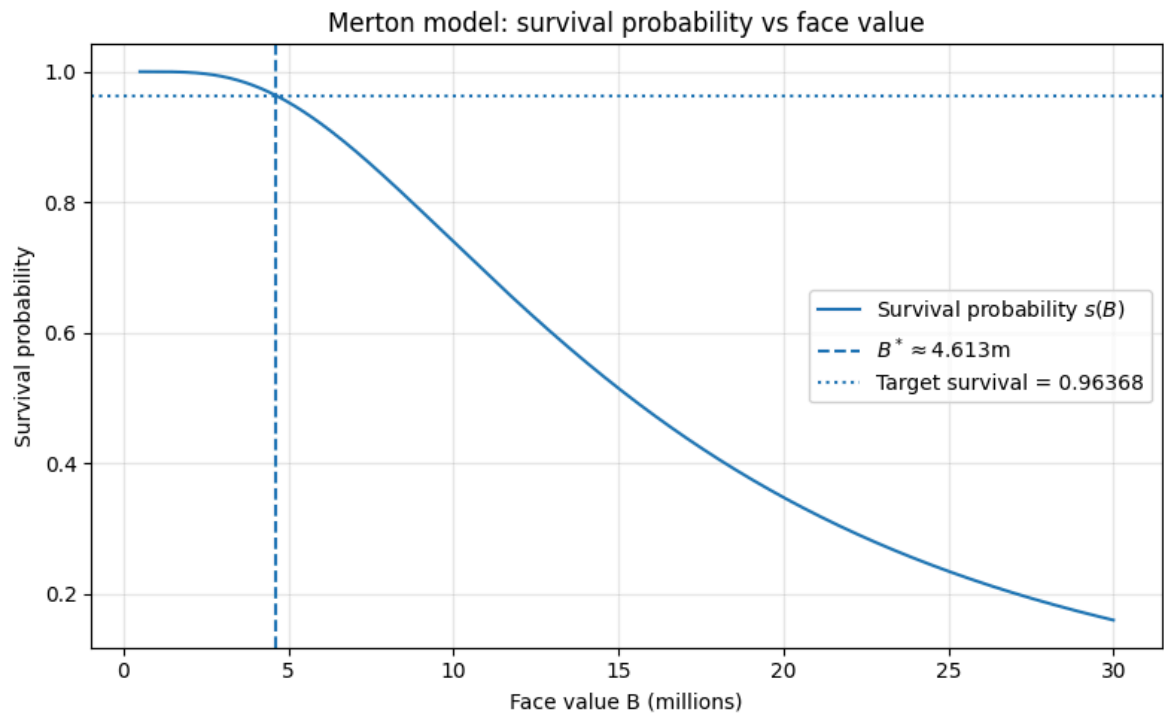
$d_2(B^*) = 1.795086$

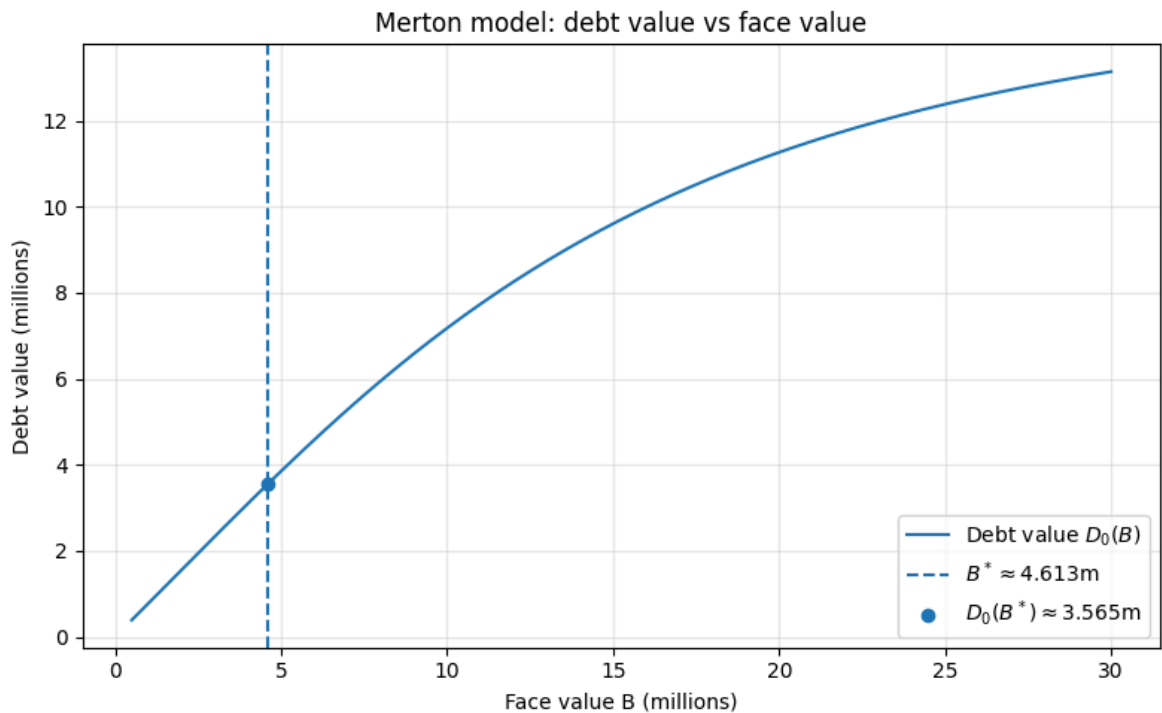
$d_1(B^*) = 2.465906$

$\text{Survival}(B^*) = 0.963680$

$\text{Equity}(B^*) = 11,435,404.64$

$\text{Debt}(B^*) = 3,564,595.36$





```
In [1]: from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.stats import norm
```

```
np.random.seed(42)
```

```
pd.set_option("display.max_columns", 50)
pd.set_option("display.width", 140)
```

```
plt.rcParams["figure.figsize"] = (12, 6)
plt.rcParams["axes.grid"] = True
```

```
In [2]: def find_project_root(start: Path, marker: str = "data") -> Path:
    p = start.resolve()
    while p != p.parent:
        if (p / marker).exists():
            return p
        p = p.parent
    raise FileNotFoundError(f"Could not find project root containing '{ma
```

```
PROJECT_ROOT = find_project_root(Path.cwd())
DATA_DIR = PROJECT_ROOT / "data"
```

```
print("PROJECT_ROOT =", PROJECT_ROOT)
print("DATA_DIR exists =", DATA_DIR.exists())
print("Files in data/ =")
for f in sorted(DATA_DIR.iterdir()):
    print(" ", f.name)
```

```
PROJECT_ROOT = /Users/nigelli/Desktop/Columbia MAFN/26Spring/MATH5320/HMWK
9
DATA_DIR exists = True
Files in data/ =
    AAPL-bloomberg.csv
    AAPL-bloomberg.xlsx
    CAT-bloomberg.csv
    CAT-bloomberg.xlsx
```

In [3]: *# — Load and align data —*

```
def load_bloomberg_csv(path: Path) -> pd.DataFrame:
    df = pd.read_csv(path)
    df.columns = [c.strip() for c in df.columns]
    return df

REQUIRED_COLS = ["Dates", "PX_LAST", "PX_OPEN", "PX_HIGH", "PX_LOW", "PX_

def prepare_price_df(df: pd.DataFrame, prefix: str) -> pd.DataFrame:
    missing = [c for c in REQUIRED_COLS if c not in df.columns]
    if missing:
        raise ValueError(f"{prefix}: missing required columns: {missing}")
    out = df[REQUIRED_COLS].copy()
    out["Dates"] = pd.to_datetime(out["Dates"])
    numeric_cols = [c for c in REQUIRED_COLS if c != "Dates"]
    for c in numeric_cols:
        out[c] = pd.to_numeric(out[c], errors="coerce")
    out = out.sort_values("Dates").dropna(subset=["Dates", "PX_LAST"])
    out = out.rename(columns={
        "PX_LAST": f"{prefix}_PX_LAST", "PX_OPEN": f"{prefix}_PX_OPEN",
        "PX_HIGH": f"{prefix}_PX_HIGH", "PX_LOW": f"{prefix}_PX_LOW",
        "PX_VOLUME": f"{prefix}_PX_VOLUME", "PX_CLOSE_1D": f"{prefix}_PX_
    })
    return out

aapl = load_bloomberg_csv(DATA_DIR / "AAPL-bloomberg.csv")
cat = load_bloomberg_csv(DATA_DIR / "CAT-bloomberg.csv")

aapl_p = prepare_price_df(aapl, "AAPL")
cat_p = prepare_price_df(cat, "CAT")

prices = (
    aapl_p.merge(cat_p, on="Dates", how="inner")
    .sort_values("Dates")
    .reset_index(drop=True)
)

print("Aligned dataset shape:", prices.shape)
display(prices.tail())
```

Aligned dataset shape: (10898, 13)

	Dates	AAPL_PX_LAST	AAPL_PX_OPEN	AAPL_PX_HIGH	AAPL_PX_LOW	AAPL_PX_CLOSE
10893	2026-02-05	275.91	278.130	279.500	273.230	275.91
10894	2026-02-06	278.12	277.120	280.905	276.925	278.12
10895	2026-02-09	274.62	277.905	278.200	271.700	274.62
10896	2026-02-10	273.68	274.885	275.370	272.940	273.68
10897	2026-02-11	275.50	274.695	280.180	274.450	275.50

```
In [4]: # — Portfolio construction (same as HW6-8) —

INITIAL_INVESTMENT = 5000
START_DATE = pd.Timestamp("1997-10-13")

start_row = prices.loc[prices["Dates"] == START_DATE]
P_AAPL_0 = start_row["AAPL_PX_LAST"].iloc[0]
P_CAT_0 = start_row["CAT_PX_LAST"].iloc[0]

N_AAPL = int(INITIAL_INVESTMENT / P_AAPL_0)
N_CAT = int(INITIAL_INVESTMENT / P_CAT_0)

print(f"Start date: {START_DATE.date()}")
print(f"AAPL price = {P_AAPL_0}, shares = {N_AAPL}")
print(f"CAT price = {P_CAT_0}, shares = {N_CAT}")

prices_work = prices.copy()
prices_work["P_value"] = N_AAPL * prices_work["AAPL_PX_LAST"] + N_CAT * p

portfolio_df = prices_work[["Dates", "AAPL_PX_LAST", "CAT_PX_LAST", "P_value"]]
portfolio_df["AAPL_ret"] = np.log(portfolio_df["AAPL_PX_LAST"] / portfolio_df["AAPL_PX_LAST"].shift(1))
portfolio_df["CAT_ret"] = np.log(portfolio_df["CAT_PX_LAST"] / portfolio_df["CAT_PX_LAST"].shift(1))
portfolio_df["P_ret"] = np.log(portfolio_df["P_value"] / portfolio_df["P_value"].shift(1))
portfolio_df = portfolio_df.dropna().reset_index(drop=True)

print(f"Portfolio dataframe: {len(portfolio_df)} rows")
display(portfolio_df.tail())
```

Start date: 1997-10-13
 AAPL price = 0.2026, shares = 24679
 CAT price = 29.9063, shares = 167
 Portfolio dataframe: 10897 rows

	Dates	AAPL_PX_LAST	CAT_PX_LAST	P_value	AAPL_ret	CAT_ret	
10892	2026-02-05	275.91	678.31	6922460.66	-0.002100	-0.019721	-C
10893	2026-02-06	278.12	726.20	6984998.88	0.007978	0.068221	0
10894	2026-02-09	274.62	742.12	6901281.02	-0.012664	0.021685	-C
10895	2026-02-10	273.68	742.37	6878124.51	-0.003429	0.000337	-C
10896	2026-02-11	275.50	775.00	6928489.50	0.006628	0.043015	0

Problem 4 — Short Portfolio VaR

Short position mechanics

Loss = Initial – Final = $V_0 - V_0(2 - e^r) = V_0(e^r - 1)$.

Short VaR formula (GBM)

Since $\text{Loss} = V_0(e^r - 1)$ is **increasing** in r , the α -quantile of loss is:

$$\text{VaR}_\alpha^{\text{short}} = V_0 \left(e^{\mu h + z_\alpha \sigma \sqrt{h}} - 1 \right)$$

where $z_\alpha = \Phi^{-1}(\alpha)$.

Compare to the **long** VaR where $\text{loss} = V_0(1 - e^r)$ is decreasing in r :

$$\text{VaR}_\alpha^{\text{long}} = V_0 \left(1 - e^{\mu h - z_\alpha \sigma \sqrt{h}} \right)$$

```
In [5]: # — Parameters —

WINDOW = 252 * 5           # 5-year rolling window
POSITION_VALUE = 10_000
N_SIMS = 10_000
HORIZON = 5                # 5 trading days

VAR_LEVEL = 0.99           # 99% VaR for Q4
ES_LEVEL = 0.975           # 97.5% ES for Q5

z_var = norm.ppf(VAR_LEVEL) # ≈ 2.326
z_es = norm.ppf(ES_LEVEL)  # ≈ 1.960
```

```
print(f"z_{{0.99}} = {z_var:.4f}")
print(f"z_{{0.975}} = {z_es:.4f}")
```

```
z_{0.99} = 2.3263
z_{0.975} = 1.9600
```

```
In [6]: # — Q4a: Formula-based 99% 5-day VaR for short positions in A, C, P —

def short_var_gbm(mu, sigma, h, V0, alpha):
    """99% VaR for a SHORT position under GBM."""
    z = norm.ppf(alpha)
    return V0 * (np.exp(mu * h + z * sigma * np.sqrt(h)) - 1)

def long_var_gbm(mu, sigma, h, V0, alpha):
    """99% VaR for a LONG position under GBM."""
    z = norm.ppf(alpha)
    return V0 * (1 - np.exp(mu * h - z * sigma * np.sqrt(h)))

start_idx = portfolio_df.index[portfolio_df["Dates"] >= START_DATE][0]

formula_rows = []

for i in range(max(start_idx, WINDOW), len(portfolio_df)):
    date_t = portfolio_df["Dates"].iloc[i]

    # 5-year rolling window
    aapl_w = portfolio_df["AAPL_ret"].iloc[i - WINDOW : i]
    cat_w = portfolio_df["CAT_ret"].iloc[i - WINDOW : i]
    p_w = portfolio_df["P_ret"].iloc[i - WINDOW : i]

    # Parameter estimates (daily)
    mu_a, sig_a = aapl_w.mean(), aapl_w.std(ddof=1)
    mu_c, sig_c = cat_w.mean(), cat_w.std(ddof=1)
    mu_p, sig_p = p_w.mean(), p_w.std(ddof=1)

    formula_rows.append({
        "Dates": date_t,
        # Short VaR (99%)
        "Short_VaR99_A": short_var_gbm(mu_a, sig_a, HORIZON, POSITION_VAL
        "Short_VaR99_C": short_var_gbm(mu_c, sig_c, HORIZON, POSITION_VAL
        "Short_VaR99_P": short_var_gbm(mu_p, sig_p, HORIZON, POSITION_VAL
        # Long VaR (99%) for comparison
        "Long_VaR99_A": long_var_gbm(mu_a, sig_a, HORIZON, POSITION_VALUE
        "Long_VaR99_C": long_var_gbm(mu_c, sig_c, HORIZON, POSITION_VALUE
        "Long_VaR99_P": long_var_gbm(mu_p, sig_p, HORIZON, POSITION_VALUE
        # Store params for Q5
        "mu_a": mu_a, "sig_a": sig_a,
        "mu_c": mu_c, "sig_c": sig_c,
        "mu_p": mu_p, "sig_p": sig_p,
    })

formula_df = pd.DataFrame(formula_rows)

print(f"Formula VaR computed for {len(formula_df)} dates")
display(formula_df[["Dates", "Short_VaR99_A", "Short_VaR99_C", "Short_VaR
                    "Long_VaR99_A", "Long_VaR99_C", "Long_VaR99_P"]].t
```

Formula VaR computed for 7127 dates

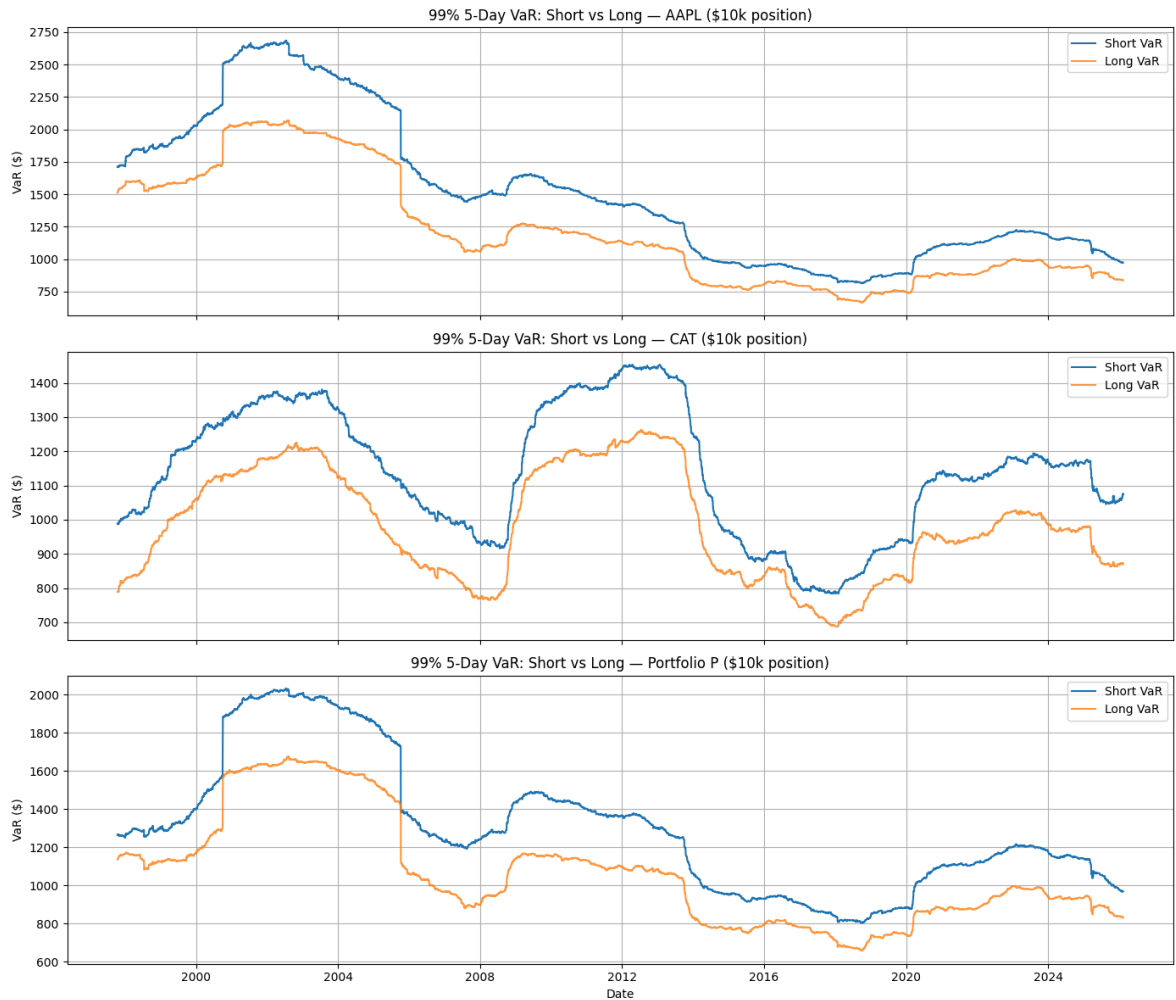
	Dates	Short_VaR99_A	Short_VaR99_C	Short_VaR99_P	Long_VaR99_A	Long
7122	2026-02-05	976.552589	1068.293333	970.023450	838.805949	
7123	2026-02-06	976.174341	1068.213880	969.636675	835.869472	8
7124	2026-02-09	975.553127	1076.217316	969.084356	835.962266	
7125	2026-02-10	974.912353	1073.893502	968.431946	836.807696	8
7126	2026-02-11	975.032278	1074.096613	968.552989	836.589292	8

```
In [7]: # — Q4b: Plots – Short VaR for A, C, P —

fig, axes = plt.subplots(3, 1, figsize=(14, 12), sharex=True)

for ax, asset in zip(axes, ["A", "C", "P"]):
    ax.plot(formula_df["Dates"], formula_df[f"Short_VaR99_{asset}"],
            label=f"Short VaR", linewidth=1.5)
    ax.plot(formula_df["Dates"], formula_df[f"Long_VaR99_{asset}"],
            label=f"Long VaR", linewidth=1.5, alpha=0.8)
    name = {"A": "AAPL", "C": "CAT", "P": "Portfolio P"}[asset]
    ax.set_title(f"99% 5-Day VaR: Short vs Long – {name} ($10k position)")
    ax.set_ylabel("VaR ($)")
    ax.legend()

axes[-1].set_xlabel("Date")
plt.tight_layout()
plt.show()
```



```
In [8]: # — Q4c: Summary table – latest date —

latest = formula_df.iloc[-1]

summary = pd.DataFrame({
    "Asset": ["AAPL (A)", "CAT (C)", "Portfolio (P)"],
    "Short 99% VaR": [latest["Short_VaR99_A"], latest["Short_VaR99_C"], latest["Short_VaR99_P"]],
    "Long 99% VaR": [latest["Long_VaR99_A"], latest["Long_VaR99_C"], latest["Long_VaR99_P"]],
})
summary["Short/Long Ratio"] = summary["Short 99% VaR"] / summary["Long 99% VaR"]

print(f"Latest date: {latest['Dates'].date()}")
display(summary)
```

Latest date: 2026-02-11

	Asset	Short 99% VaR	Long 99% VaR	Short/Long Ratio
0	AAPL (A)	975.032278	836.589292	1.165485
1	CAT (C)	1074.096613	872.268782	1.231383
2	Portfolio (P)	968.552989	830.553179	1.166154

Why short positions are riskier

Short VaR > Long VaR because:

1. **Loss asymmetry under GBM:** Short loss = $V_0(e^r - 1)$ is convex and unbounded above, while long loss = $V_0(1 - e^r)$ is bounded above by V_0 .
2. **Positive drift:** Under GBM with positive expected returns, the stock is more likely to go up, which *hurts* the short.
3. **Jensen's inequality:** The exponential in the short loss formula amplifies upward moves more than the long formula amplifies downward moves.

```
In [9]: # — Q4d: Monte Carlo VaR for short P by simulating A and C —

def compute_var_es(losses, var_level=0.99, es_level=0.975):
    var_value = np.quantile(losses, var_level)
    es_cutoff = np.quantile(losses, es_level)
    es_value = losses[losses >= es_cutoff].mean()
    return var_value, es_value

def mc_short_underlying(
    aapl_returns_window, cat_returns_window,
    s_aapl_t, s_cat_t, weight_aapl_t, weight_cat_t,
    position_value=10_000, horizon_days=5, n_sims=10_000
):
    """MC VaR/ES for a SHORT portfolio position, simulating the two under
    X = np.column_stack([aapl_returns_window.values, cat_returns_window.v
    mu = X.mean(axis=0)
    cov = np.cov(X.T, ddof=1)

    mu_h = mu * horizon_days
    cov_h = cov * horizon_days

    sim_r = np.random.multivariate_normal(mu_h, cov_h, size=n_sims)

    sim_aapl = s_aapl_t * np.exp(sim_r[:, 0])
    sim_cat = s_cat_t * np.exp(sim_r[:, 1])

    # Number of shares for a $10k SHORT position
    n_aapl = (position_value * weight_aapl_t) / s_aapl_t
    n_cat = (position_value * weight_cat_t) / s_cat_t

    # Short portfolio value at horizon
    sim_port_value = n_aapl * sim_aapl + n_cat * sim_cat

    # SHORT loss = portfolio appreciated → we lose
    losses = sim_port_value - position_value

    var99, es975 = compute_var_es(losses, 0.99, 0.975)
    return var99, es975

mc_rows = []

for i in range(max(start_idx, WINDOW), len(portfolio_df)):
    date_t = portfolio_df["Dates"].iloc[i]

    aapl_w = portfolio_df["AAPL_ret"].iloc[i - WINDOW : i]
    cat_w = portfolio_df["CAT_ret"].iloc[i - WINDOW : i]

    s_a = portfolio_df["AAPL_PX_LAST"].iloc[i]
    s_c = portfolio_df["CAT_PX_LAST"].iloc[i]
```

```

p_t = portfolio_df["P_value"].iloc[i]

w_a = (N_AAPL * s_a) / p_t
w_c = (N_CAT * s_c) / p_t

var99, es975 = mc_short_underlying(
    aapl_w, cat_w, s_a, s_c, w_a, w_c,
    position_value=POSITION_VALUE, horizon_days=HORIZON, n_sims=N_SIM
)

mc_rows.append({
    "Dates": date_t,
    "MC_Short_VaR99_P": var99,
    "MC_Short_ES975_P": es975,
})

mc_df = pd.DataFrame(mc_rows)

print(f"MC Short VaR computed for {len(mc_df)} dates")
display(mc_df.tail())

```

MC Short VaR computed for 7127 dates

	Dates	MC_Short_VaR99_P	MC_Short_ES975_P
7122	2026-02-05	974.920721	972.704875
7123	2026-02-06	972.785654	988.630021
7124	2026-02-09	921.608116	920.293917
7125	2026-02-10	936.850895	962.963914
7126	2026-02-11	962.861797	971.783543

```

In [10]: # — Q4e: Compare formula-P vs MC(A,C) for short VaR —

compare_df = formula_df[["Dates", "Short_VaR99_P"]].merge(mc_df, on="Date")

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(compare_df["Dates"], compare_df["Short_VaR99_P"],
        label="Formula (P returns)", linewidth=1.5)
ax.plot(compare_df["Dates"], compare_df["MC_Short_VaR99_P"],
        label="MC (simulate A & C)", linewidth=1.5, alpha=0.8)
ax.set_title("Short 99% 5-Day VaR on P: Formula vs Monte Carlo")
ax.set_ylabel("VaR ($)")
ax.set_xlabel("Date")
ax.legend()
plt.tight_layout()
plt.show()

# Zoomed view
zoom = compare_df.loc[compare_df["Dates"] >= "2018-01-01"]

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(zoom["Dates"], zoom["Short_VaR99_P"],
        label="Formula (P returns)", linewidth=2)
ax.plot(zoom["Dates"], zoom["MC_Short_VaR99_P"],
        label="MC (simulate A & C)", linewidth=2, alpha=0.8)
ax.set_title("Short 99% 5-Day VaR on P: Formula vs Monte Carlo (Zoomed)")
ax.set_ylabel("VaR ($)")

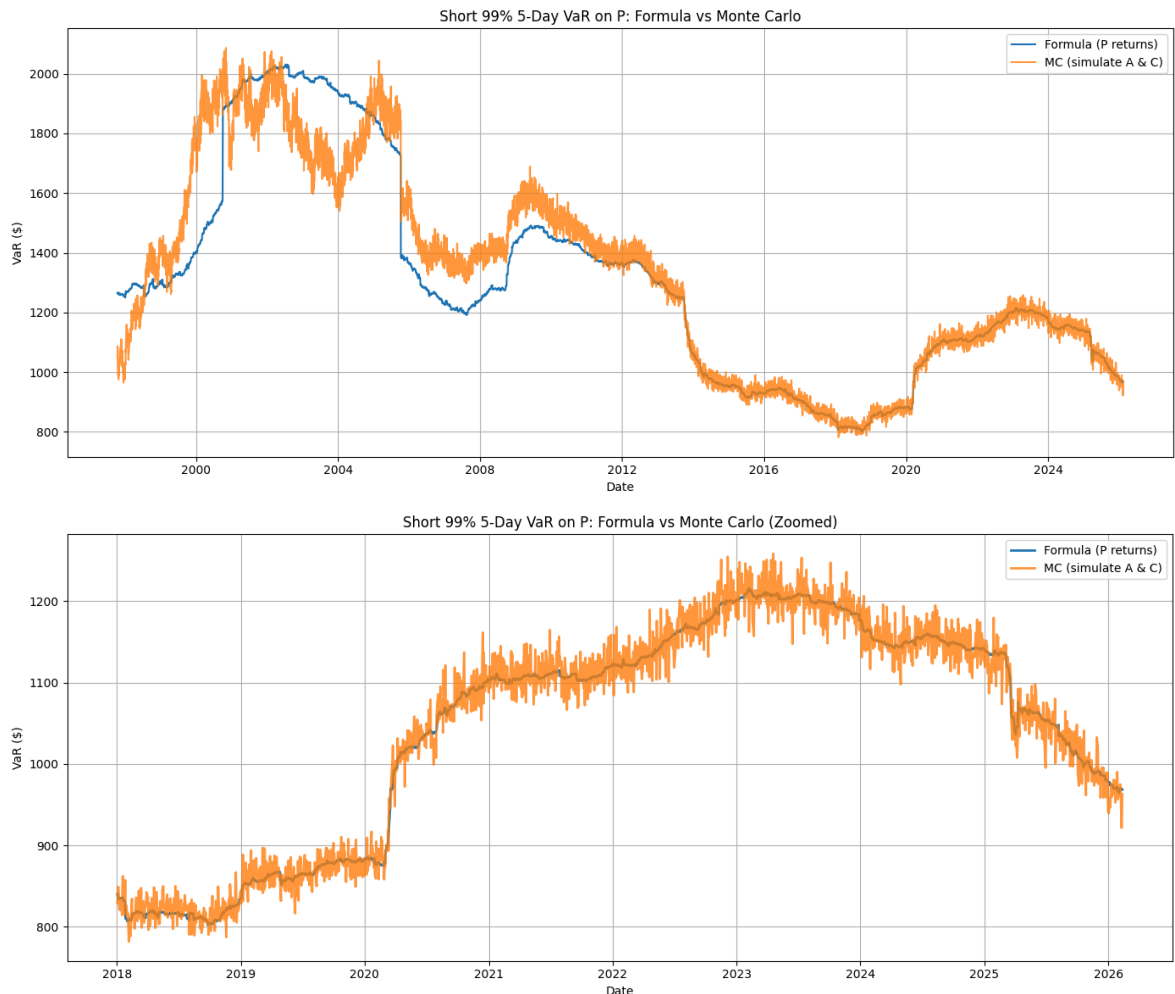
```

```

ax.set_xlabel("Date")
ax.legend()
plt.tight_layout()
plt.show()

# Latest comparison
latest_c = compare_df.iloc[-1]
print(f"\nLatest date: {latest_c['Dates'].date()}")
print(f"  Formula (P):  ${latest_c['Short_VaR99_P']:.2f}")
print(f"  MC (A & C):    ${latest_c['MC_Short_VaR99_P']:.2f}")

```



Latest date: 2026-02-11

Formula (P): \$968.55

MC (A & C): \$962.86

Formula P vs MC(A,C) comparison

The formula approach treats P as a single GBM, estimating μ and σ directly from P's log returns. The MC approach simulates the two underlyings as correlated GBMs and reconstructs P.

The MC approach captures the *nonlinear* composition of P from A and C (since portfolio weights drift over time), whereas the formula assumes P itself is exactly lognormal. Since the portfolio is a *weighted sum* of lognormals (not a product), P is not exactly lognormal — the MC is more accurate, though the formula is a reasonable approximation when correlations and weight drift are moderate.

Problem 5 — Short Portfolio, Formula ES

```
In [11]: # — Q5: Formula ES for short positions —

def short_es_gbm(mu, sigma, h, V0, alpha):
    """97.5% ES for a SHORT position under GBM."""
    z = norm.ppf(alpha)
    numerator = np.exp(mu * h + sigma**2 * h / 2) * norm.cdf(sigma * np.s
    return V0 * (numerator / (1 - alpha) - 1)

formula_df["Short_ES975_A"] = formula_df.apply(
    lambda r: short_es_gbm(r["mu_a"], r["sig_a"], HORIZON, POSITION_VALUE
formula_df["Short_ES975_C"] = formula_df.apply(
    lambda r: short_es_gbm(r["mu_c"], r["sig_c"], HORIZON, POSITION_VALUE
formula_df["Short_ES975_P"] = formula_df.apply(
    lambda r: short_es_gbm(r["mu_p"], r["sig_p"], HORIZON, POSITION_VALUE

display(formula_df[["Dates", "Short_VaR99_A", "Short_ES975_A",
                    "Short_VaR99_C", "Short_ES975_C",
                    "Short_VaR99_P", "Short_ES975_P"]].tail())
```

	Dates	Short_VaR99_A	Short_ES975_A	Short_VaR99_C	Short_ES975_C	Sho
7122	2026-02-05	976.552589	982.413195	1068.293333	1074.654340	
7123	2026-02-06	976.174341	982.021315	1068.213880	1074.577848	
7124	2026-02-09	975.553127	981.398012	1076.217316	1082.624264	
7125	2026-02-10	974.912353	980.758178	1073.893502	1080.288807	
7126	2026-02-11	975.032278	980.877678	1074.096613	1080.491670	

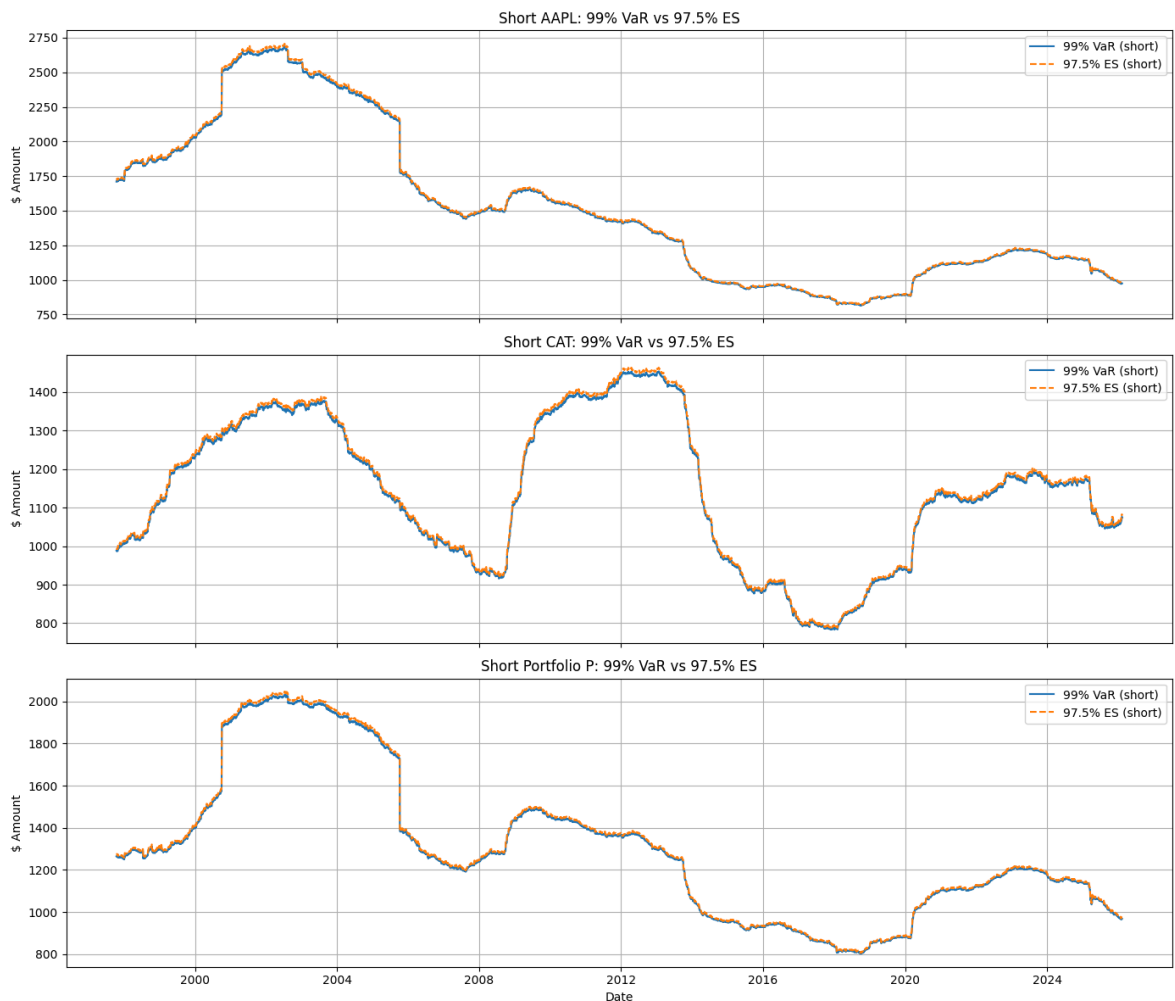
```
In [12]: # — Q5: Compare 99% VaR vs 97.5% ES for short positions —

fig, axes = plt.subplots(3, 1, figsize=(14, 12), sharex=True)

for ax, asset in zip(axes, ["A", "C", "P"]):
    ax.plot(formula_df["Dates"], formula_df[f"Short_VaR99_{asset}"],
            label="99% VaR (short)", linewidth=1.5)
    ax.plot(formula_df["Dates"], formula_df[f"Short_ES975_{asset}"],
            label="97.5% ES (short)", linewidth=1.5, linestyle="--")
    name = {"A": "AAPL", "C": "CAT", "P": "Portfolio P"}[asset]
    ax.set_title(f"Short {name}: 99% VaR vs 97.5% ES")
    ax.set_ylabel("$ Amount")
    ax.legend()

axes[-1].set_xlabel("Date")
```

```
plt.tight_layout()
plt.show()
```



```
In [13]: # — Q5: Summary table —

latest = formula_df.iloc[-1]

es_summary = pd.DataFrame({
    "Asset": ["AAPL (A)", "CAT (C)", "Portfolio (P)"],
    "Short 99% VaR": [latest["Short_VaR99_A"], latest["Short_VaR99_C"],
    "Short 97.5% ES": [latest["Short_ES975_A"], latest["Short_ES975_C"],
    })
es_summary["ES/VaR Ratio"] = es_summary["Short 97.5% ES"] / es_summary["S

print(f"Latest date: {latest['Dates'].date()}")
display(es_summary)
```

Latest date: 2026-02-11

	Asset	Short 99% VaR	Short 97.5% ES	ES/VaR Ratio
0	AAPL (A)	975.032278	980.877678	1.005995
1	CAT (C)	1074.096613	1080.491670	1.005954
2	Portfolio (P)	968.552989	974.347772	1.005983

```
In [14]: # — Q5: Time series of ratio —

for asset in ["A", "C", "P"]:
```

```

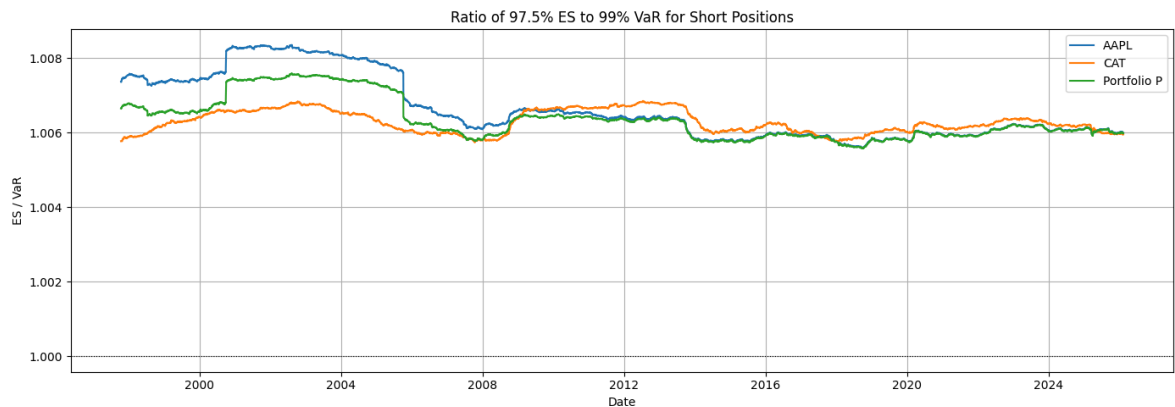
formula_df[f"ES_VaR_Ratio_{asset}"] = (
    formula_df[f"Short_ES975_{asset}"] / formula_df[f"Short_VaR99_{as
)

fig, ax = plt.subplots(figsize=(14, 5))
for asset, name in [("A", "AAPL"), ("C", "CAT"), ("P", "Portfolio P")]:
    ax.plot(formula_df["Dates"], formula_df[f"ES_VaR_Ratio_{asset}"],
            label=name, linewidth=1.5)

ax.axhline(1.0, color="black", linestyle=":", linewidth=0.8)
ax.set_title("Ratio of 97.5% ES to 99% VaR for Short Positions")
ax.set_ylabel("ES / VaR")
ax.set_xlabel("Date")
ax.legend()
plt.tight_layout()
plt.show()

print("\nSummary statistics for ES/VaR ratio:")
for asset, name in [("A", "AAPL"), ("C", "CAT"), ("P", "Portfolio")]:
    r = formula_df[f"ES_VaR_Ratio_{asset}"]
    print(f" {name:12s} - mean: {r.mean():.4f}, std: {r.std():.4f}, "
          f"min: {r.min():.4f}, max: {r.max():.4f}")

```



Summary statistics for ES/VaR ratio:

AAPL	– mean: 1.0066, std: 0.0008, min: 1.0056, max: 1.0084
CAT	– mean: 1.0063, std: 0.0003, min: 1.0057, max: 1.0068
Portfolio	– mean: 1.0064, std: 0.0006, min: 1.0056, max: 1.0076